

Consenso distribuito

Il protocollo Raft

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

21 maggio 2026



UniMe
1548

Inconsistenza in un sistema replicato

Conto bancario con 100, replicato su cinque server.

Client A e Client B vogliono entrambi prelevare 80 quasi contemporaneamente.

- ▶ Server 1 riceve A: saldo 100, approva
- ▶ Server 2 riceve B *nello stesso istante*: vede ancora 100 (l'aggiornamento di S1 non è ancora arrivato), approva

Risultato

A e B ottengono 80 ciascuno.

Totale prelevato: **160**

Disponibile: **100**

I server mostrano saldi diversi.

Il sistema è **inconsistente**.

La radice del problema

Senza coordinamento, più server possono prendere decisioni incompatibili, ciascuno ragionando sulla propria visione locale, già obsoleta.

Eleggere uno dei server come **leader**. Tutte le operazioni dei client passano per lui.

Prima di eseguire qualsiasi operazione, il leader la registra in un elenco numerato — il **log** — e lo distribuisce agli altri server.

Solo quando la **maggioranza** ha confermato, il leader esegue e risponde al client.

Perché si chiama Raft?

Raft = zattera: tronchi legati insieme che si muovono come un blocco unico. I server fanno la stessa cosa.

Tornando all'esempio:

Client A chiede di prelevare 80. Il leader scrive nel log “op 1: preleva 80” e lo manda agli altri 4 server. Appena 3 su 5 confermano, il leader esegue. Saldo: 20.

Arriva la richiesta di B: saldo = 20 $\Rightarrow$ rifiuta.

Proprietà fondamentale

Tutti i server che eseguono le stesse operazioni nello stesso ordine arrivano necessariamente allo stesso stato.

I tre meccanismi di Raft

1. Elezione del leader

Uno dei server si candida, raccoglie i voti della maggioranza e diventa leader. Se crasha, i follower avviano una nuova elezione.

2. Replicazione del log

Le operazioni arrivano al leader, che le replica sui follower. Solo dopo la conferma della maggioranza l'operazione è *committed*.

3. Sicurezza dopo crash

I follower si allineano sempre al leader, mai il contrario.

I tre meccanismi di Raft



I tre meccanismi formano un ciclo. Il sistema non si ferma mai finché la maggioranza dei server è raggiungibile.

I due messaggi del protocollo

Raft usa solo **due tipi di messaggi**. Heartbeat, richieste di voto, replicazione e riconciliazione del log sono tutti casi particolari di questi due.

RequestVote — richiesta di voto

Inviato dal **candidate** a tutti.

<code>term</code>	term del candidato
<code>candidateId</code>	chi si candida
<code>lastLogIndex</code>	lunghezza del log
<code>lastLogTerm</code>	term ultima entry

Risposta: `currentTerm` + `voteGranted`

AppendEntries — log e heartbeat

Inviato dal **leader** a tutti i follower.

<code>term</code>	term del leader
<code>leaderId</code>	chi è il leader
<code>prevLogIndex</code>	entry precedente
<code>prevLogTerm</code>	term entry precedente
<code>entries[]</code>	nuove entry (vuoto = hb)
<code>leaderCommit</code>	commitIndex del leader

Risposta: `currentTerm` + `success`

Ruoli e struttura del tempo

Ogni server si trova sempre in uno di tre stati:

Follower

Stato normale. Riceve heartbeat dal leader. Non prende iniziative.

Candidate

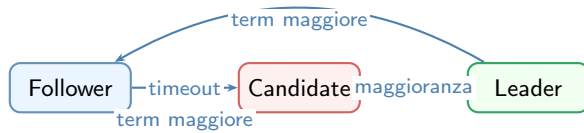
Stato transitorio. Si attiva quando mancano gli heartbeat. Chiede voti agli altri server.

Leader

Uno solo per volta. Riceve comandi dai client e replica il log sui follower.

Il tempo è diviso in **term** — mandati numerati progressivamente.

- ▶ Ogni term inizia con un'elezione
- ▶ Ha al massimo un leader



Il leader manda periodicamente `AppendEntries` vuoti (**heartbeat**). Se un follower non ne riceve uno entro il **timeout**, presume che il leader sia morto e si candida:

1. Incrementa il proprio term
2. Vota per sé stesso
3. Manda `RequestVote` a tutti

Un server concede il voto solo se il log del candidato è **almeno aggiornato quanto il proprio**:

- ▶ Prima si confronta `lastLogTerm`: chi ha il term più alto vince
- ▶ A parità di term, vince chi ha il log con più entries

Vince chi arriva primo

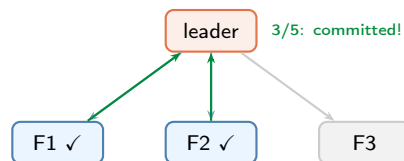
Ogni server vota **una sola volta per term**. Il primo `RequestVote` valido riceve il voto; il secondo candidato lo trova già assegnato e viene rifiutato.

Timeout casuale

Se due candidati si presentano simultaneamente e nessuno raggiunge la maggioranza, si aspetta un timeout *casuale* prima di riprovare. Statisticamente uno si sveglia prima e vince indisturbato.

Quando il leader riceve un'operazione dal client:

1. La aggiunge al proprio log come entry **pendente**
2. La manda in parallelo a tutti i follower tramite `AppendEntries`
3. Quando la **maggioranza** risponde con successo $\Rightarrow$ entry **committed**
4. Esegue l'operazione e risponde al client
5. Comunica `leaderCommit` ai follower: anche loro eseguono fino a quell'indice



F3 può essere lento o crashato.
Non serve: 3 su 5 bastano.

Il leader risponde al client **solo dopo** il commit.

La consistenza dopo un crash

Dopo un crash, il log del nuovo leader potrebbe non essere identico a quello di tutti i follower.

Regola: i follower si allineano sempre al leader, *mai il contrario*.

Come funziona:

1. Il leader manda `AppendEntries` con `prevLogIndex` e `prevLogTerm`
2. Il follower verifica di avere quella entry in quella posizione
3. Se non la trova $\Rightarrow$ rifiuta
4. Il leader arretra di uno e riprova
5. Trovato il punto di coincidenza, manda tutte le entry mancanti, sovrascrivendo quelle conflittuali

Cosa viene sovrascritto?

Solo entry **pending**, mai committed. Se un'entry fosse stata committed, sarebbe già nel log del nuovo leader (grazie alla condizione sul voto) e non verrebbe mai toccata.

È sempre il leader che parla per primo

I follower non cancellano nulla spontaneamente. Reagiscono alle `AppendEntries` del leader: se rilevano un conflitto, troncano e riscrivono.